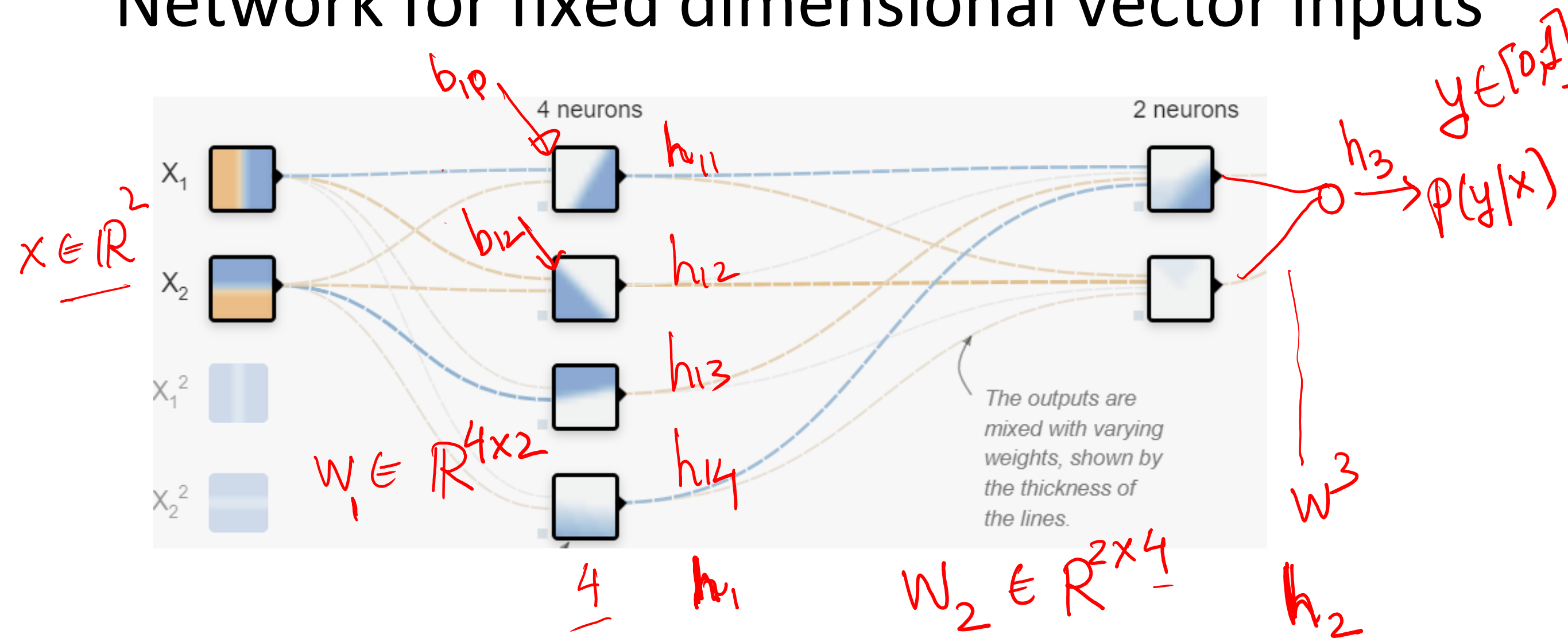


Some example networks

Network for fixed dimensional vector inputs



Input: x

$$h_1 = g(w_1 x + b_1) ; b_1 \in \mathbb{R}^4$$

$$h_2 = g(w_2 h_1 + b_2) ; b_2 \in \mathbb{R}^2$$

$$h_3 = g(w_3 h_2 + b_3) ; b_3 \in \mathbb{R}$$

$$p(y|x) = \text{Ber}(y | \sigma(h_3))$$

MLP for Image Classification

Flatten image into a single dimensional vector and apply network similar to above. Demo link: https://colab.research.google.com/drive/1Ylf3n-b4s_Dj55dEzubTln313bDxHmNy?usp=sharing

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 10)	1290

Total params: 118,282
Trainable params: 118,282
Non-trainable params: 0

Table 13.2: Structure of the MLP used for MNIST classification. Note that $100,480 = (784 + 1) \times 128$, and $16,512 = (128 + 1) \times 128$. [mlp_mnist_tf.ipynb](#).

$$x \in \mathbb{R}^{784} \quad 4096 = 28 \times 28$$

$$h_1 = g(w_1 x + b_1)$$

$$h_2 = g(w_2 h_1 + b_2)$$

$$\sim (128 \times 128 + 128)$$

$$h_3 = w_3 h_2 + b_3$$

$$P(y|x) = \text{softmax}(h_3)$$

$$w_1 \in \mathbb{R}^{128 \times 784}$$

$$b_1 \in \mathbb{R}^{128}$$

$$w_2 \in \mathbb{R}^{128 \times 128}$$

$$b_2 \in \mathbb{R}^{128}$$

$$w_3 \in \mathbb{R}^{10 \times 128}$$

$$b_3 \in \mathbb{R}^{10}$$

$$= e^{h_{3y}}$$

$$\sum_{j=1}^{10} e^{h_{3j}}$$

MLP for text classification

Text is a sequence of words, and could be of variable length. We will see how we handle this

Input x : $v_1, v_2, \dots, v_T$ where T is variable
Each $v_j \in \mathcal{V} \leftarrow$ vocabulary of words. $|\mathcal{V}| = M$
 v_j is encoded as a 1-hot vector of size M .

- ① First layer converts each word into a E dimensional real-vector called the embedding
for $t = 1$ to T : $e_t = W_1 v_t$ $W_1 \in \mathbb{R}^{E \times M}$
- ② Pool the embedding of the words

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 16)	160000
global_average_pooling1d (Gl	(None, 16)	0
dense (Dense)	(None, 16)	272
dense_1 (Dense)	(None, 1)	17

Total params: 160,289

Trainable params: 160,289

Non-trainable params: 0

Table 13.3: Structure of the MLP used for IMDB review classification. We use a vocabulary size of $V = 10,000$, an embedding size of $E = 16$, and a hidden layer of size 16. The embedding matrix $\mathbf{W}_1$ has size $10,000 \times 16$, the hidden layer (labeled “dense”) has a weight matrix $\mathbf{W}_2$ of size 16×16 and bias $\mathbf{b}_2$ of size 16 (note that $16 \times 16 + 16 = 272$), and the final layer (labeled “dense_1”) has a weight vector $\mathbf{w}_3$ of size 16 and a bias b_3 of size 1. The global average pooling layer has no free parameters. [mlp_imdb_tf.ipynb](#).

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}_3^\top \mathbf{h} + b_3))$$

$$\mathbf{h} = \varphi(\mathbf{W}_2 \bar{\mathbf{e}} + \mathbf{b}_2)$$

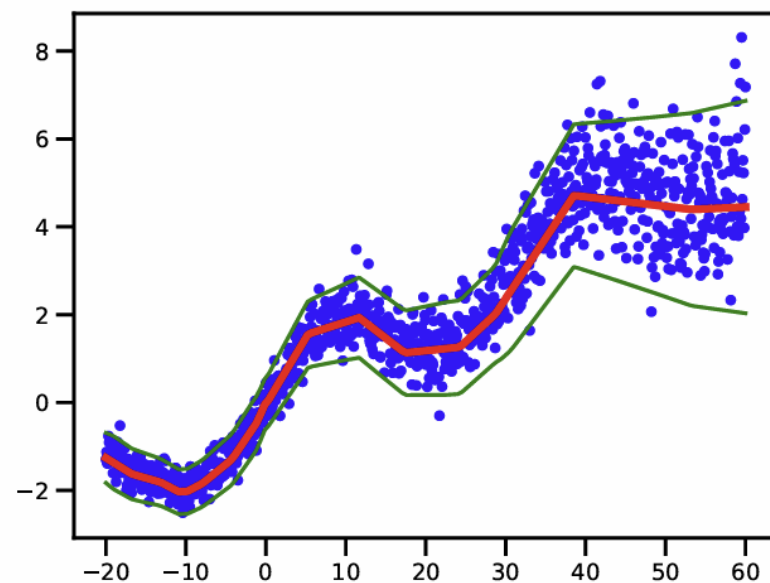
$$\bar{\mathbf{e}} = \frac{1}{T} \sum_{t=1}^T \mathbf{e}_t$$

$$\mathbf{e}_t = \mathbf{W}_1 \mathbf{v}_t$$

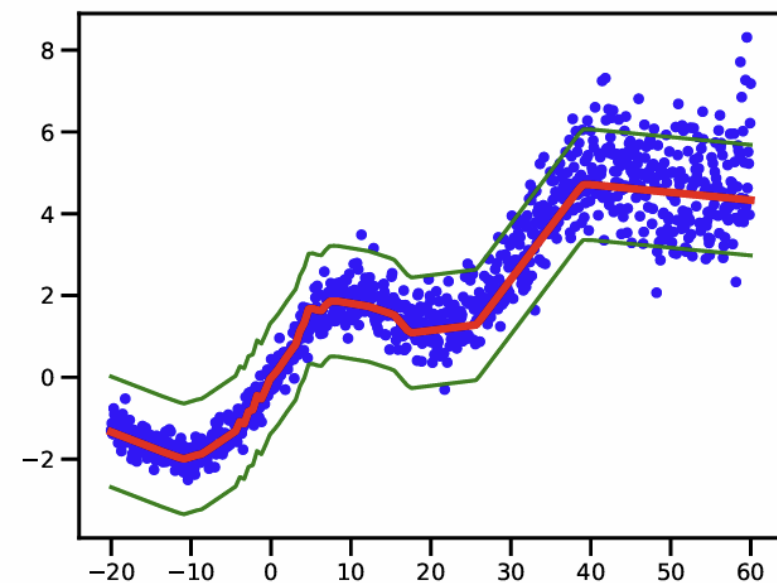
Please follow 13.2.4.3 of PML book. My notes got discarded.

MLP for heteroskedastic regression

- Variance of the regression function is not constant (like we assumed for linear regression) but dependent on x . NN allows to train a model that outputs both mean and variance as a function of x .



(a)



(b)

Figure 13.6: Illustration of predictions from an MLP fit using MLE to a 1d regression dataset with growing noise. (a) Output variance is input-dependent, as in Figure 13.5. (b) Mean is computed using same model as in (a), but output variance is treated as a fixed parameter σ^2 , which is estimated by MLE after training, as in Section 11.2.3.6. Generated by `mlp_1d_regression_hetero_tfp.ipynb`.

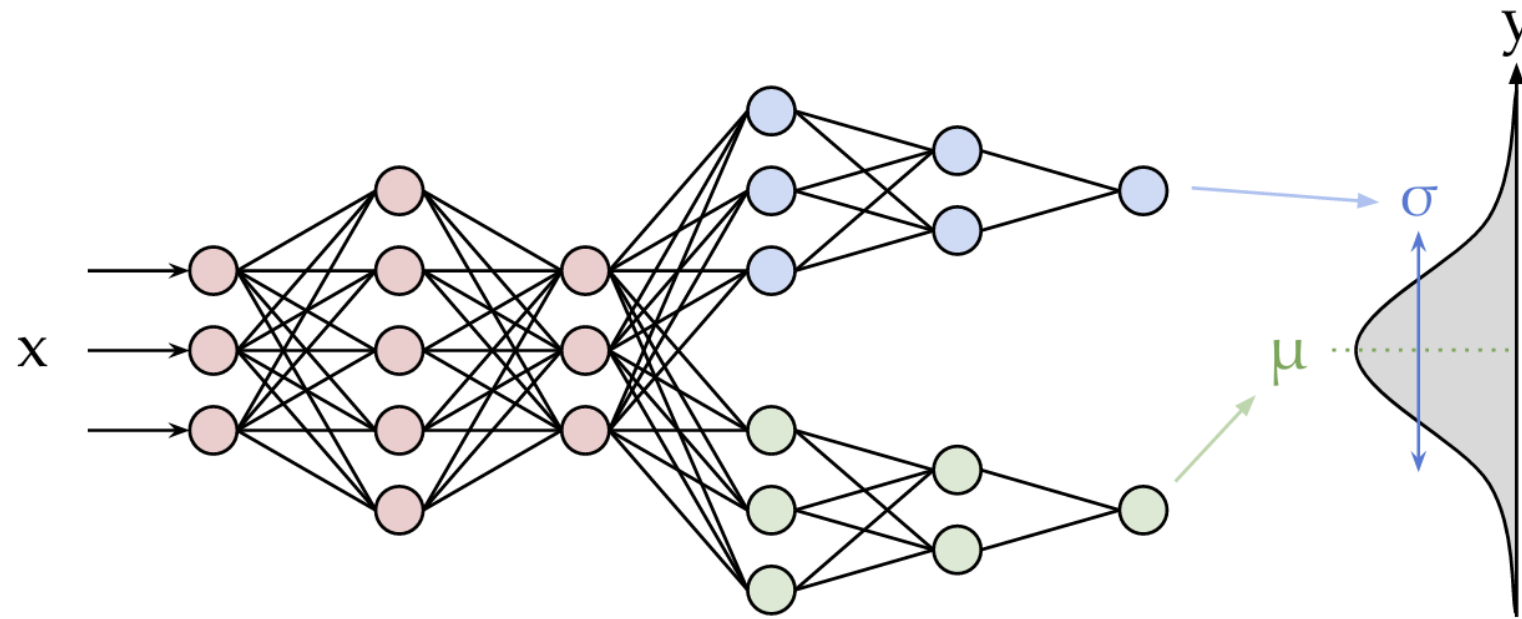


Figure 13.5: Illustration of an MLP with a shared “backbone” and two output “heads”, one for predicting the mean and one for predicting the variance. From <https://brendanhasz.github.io/2019/07/23/bayesian-density-net.html>. Used with kind permission of Brendan Hasz.

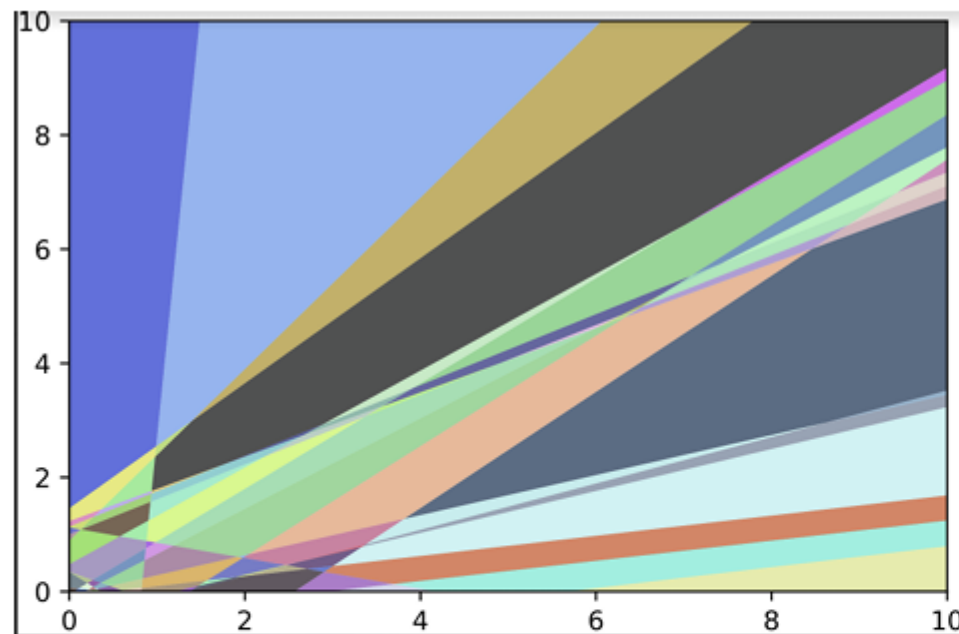
Please follow 13.2.4.4 of PML book. My notes got discarded.

Network Architecture

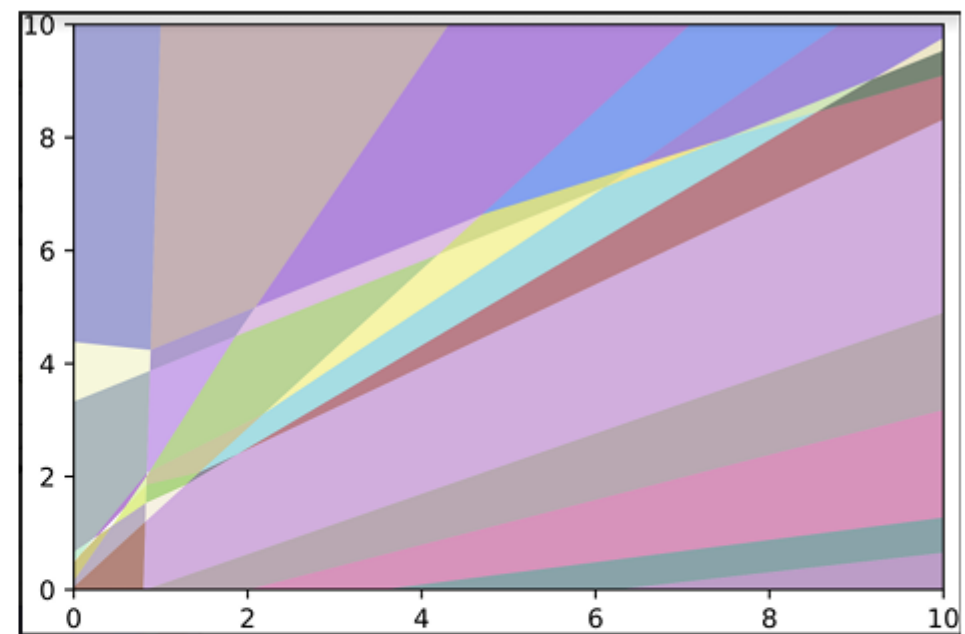
Choosing the number of layers and width of the network and connection between layer

- **Universal approximation theorem:** A network with one hidden layer (sigmoid type activation) can approximate any continuous function from a closed and bounded set given enough hidden units.
- Proof also extended to work for RELU activations.
Intuitive proof: Each hidden unit defines a half plane and a collection of them can carve out any region to which we want to assign a label.

Network Architecture



(a)



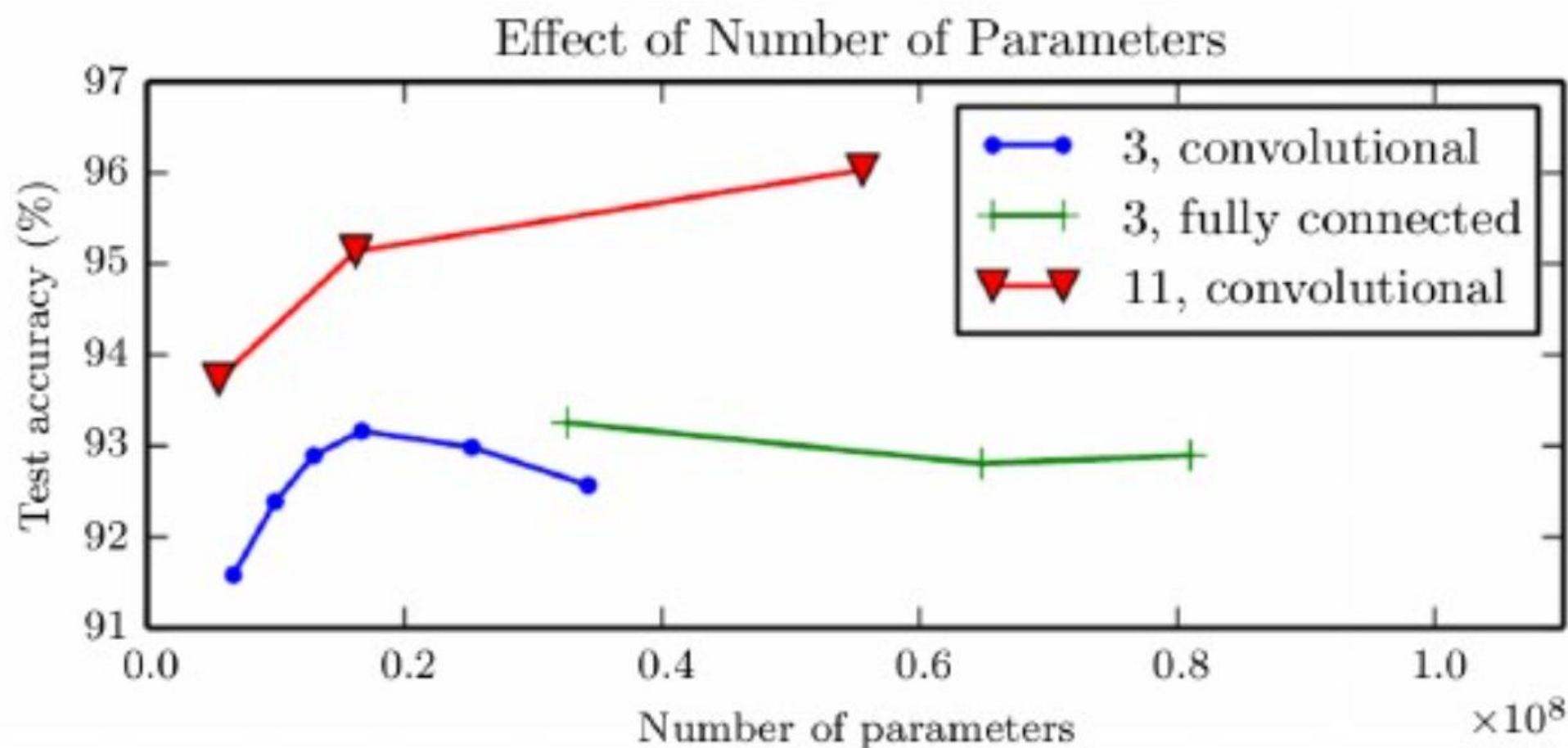
(b)

Figure 13.7: A decomposition of $\mathbb{R}^2$ into a finite set of linear decision regions produced by an MLP with ReLU activations with (a) one hidden layer of 25 hidden units and (b) two hidden layers. From Figure 1 of [HAB19]. Used with kind permission of Maksym Andriuschenko.

Not useful in practice: number of hidden units required may be exponentially large, the parameters of the network may not be easily learnable: might overfit on a wrong function.

Effect of depth

- Many functions can be efficiently represented with multiple hidden layers but require exponential width with single hidden layer
- The number of linear regions carved out via d inputs, $l+1$ depth, c units per hidden layer is $O(C(c, d)^{dl} c^d)$
- Empirically too, larger depth leads to better generalization and lower error.



Training a Feed-forward network

- To train a neural network, define a loss function $L(y, \tilde{y})$:
a function of the true output y and the predicted output $\tilde{y}$
- $L(y, \tilde{y})$ assigns a non-negative numerical score to the neural network's output, $\tilde{y}$
- The parameters of the network are set to minimise L over the training examples (i.e. a sum of losses over different training samples)
- L is typically minimised using a *gradient-based method*, in particular *stochastic gradient descent*.

Computing gradients

Modern machine learning libraries come packaged with software for automatically computing gradients.

However, to get a deeper understanding of the working of neural networks we will study the well-known backpropagation algorithm for computing the gradients manually.

Backpropagation

Please follow 13.3.2 and 13.3.3 of PML book.
My notes got discarded.

Algorithm 13.3: Backpropagation for an MLP with K layers

```
1 // Forward pass
2  $\mathbf{x}_1 := \mathbf{x}$ 
3 for  $k = 1 : K$  do
4    $\mathbf{x}_{k+1} = \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)$ 
5 // Backward pass
6  $\mathbf{u}_{K+1} := 1$ 
7 for  $k = K : 1$  do
8    $\mathbf{g}_k := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \boldsymbol{\theta}_k}$ 
9    $\mathbf{u}_k^\top := \mathbf{u}_{k+1}^\top \frac{\partial \mathbf{f}_k(\mathbf{x}_k, \boldsymbol{\theta}_k)}{\partial \mathbf{x}_k}$ 
10 // Output
11 Return  $\mathcal{L} = \mathbf{x}_{K+1}$ ,  $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{u}_1$ ,  $\{\nabla_{\boldsymbol{\theta}_k} \mathcal{L} = \mathbf{g}_k : k = 1 : K\}$ 
```
